ELEC2301 Machine Learning Assignment: Agricultural Data Analysis and Battery Temperature Prediction

Part A: Agricultural Pattern Analysis and Crop Prediction

Objective: In the present section, we analyse the agricultural soil and climatic features to cluster similar farming zones, and to forecast the crop. Clustering algorithms are first applied in order to unveil clusters in the data set, and then classification algorithms are trained to predict suitable crop labels from the given features.

"The dataset was checked for missing values and consistency before modelling. Standardisation was applied to ensure that all features contribute equally to clustering, as variables such as rainfall and nutrient levels have different scales."

In [1]:
```
# ================================================
# Part-A: Machine Learning for Precision Agriculture:
# ================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, accuracy_score, classification_report
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
import warnings

plt.style.use('seaborn-v0_8-whitegrid')
warnings.filterwarnings('ignore')

# load the dataset
df_crops = pd.read_csv('Crops_data.csv')

# Clean up column names just in case there are invisible spaces:
df_crops.columns = df_crops.columns.str.strip()
```

```
# drop the fertilizer name to prevent the data leakage:
df_crops = df_crops.drop('Fertilizer Name', axis=1)

# convert 'Soil Type' text into numbers:
df_crops = pd.get_dummies(df_crops, columns=['Soil Type'])
```

In [2]:
```
# display basic information:
print("Dataset Shape:", df_crops.shape)
print("\n--- Data Types & Missing Values ---")
print(df_crops.info())

display(df_crops.head())
```

```
Dataset Shape: (8000, 12)

--- Data Types & Missing Values ---
<class 'pandas.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Temperature       8000 non-null   float64
 1   Humidity          8000 non-null   float64
 2   Moisture          8000 non-null   float64
 3   Crop Type         8000 non-null   str
 4   Nitrogen          8000 non-null   int64
 5   Potassium         8000 non-null   int64
 6   Phosphorous       8000 non-null   int64
 7   Soil Type_Black   8000 non-null   bool
 8   Soil Type_Clayey  8000 non-null   bool
 9   Soil Type_Loamy   8000 non-null   bool
 10  Soil Type_Red     8000 non-null   bool
 11  Soil Type_Sandy   8000 non-null   bool
dtypes: bool(5), float64(3), int64(3), str(1)
memory usage: 476.7 KB
None
```

	Temperature	Humidity	Moisture	Crop Type	Nitrogen	Potassium	Phosphorous	Soil Type_Black	Soil Type_Clayey	Soil Type_Loamy	Soil Type_Red	Soil Type_Sandy
0	26.0	52.0	38.0	Maize	37	0	0	False	False	False	False	True
1	29.0	52.0	45.0	Sugarcane	12	0	36	False	False	True	False	False
2	34.0	65.0	62.0	Cotton	7	9	30	True	False	False	False	False
3	32.0	62.0	34.0	Tobacco	22	0	20	False	False	False	True	False
4	28.0	54.0	46.0	Paddy	35	0	0	False	True	False	False	False

1. Exploratory Data Analysis (EDA) of Agricultural Features

This is the preliminary step in analysing the agricultural data set before we apply machine learning. This is done to gain insight into the distribution of variables like nitrogen, phosphorus, potassium, temperature, humidity, pH and rainfall, and to see how these variables are related to various crop types.

In [3]:
```
# ========================================
# 1. Exploratory Data Analysis (EDA) & Setup
# ========================================

# 1.1 descriptive statistics to understand ranges and distributions:
display(df_crops.describe().T)
print("\n--- Feature Engineering ---")

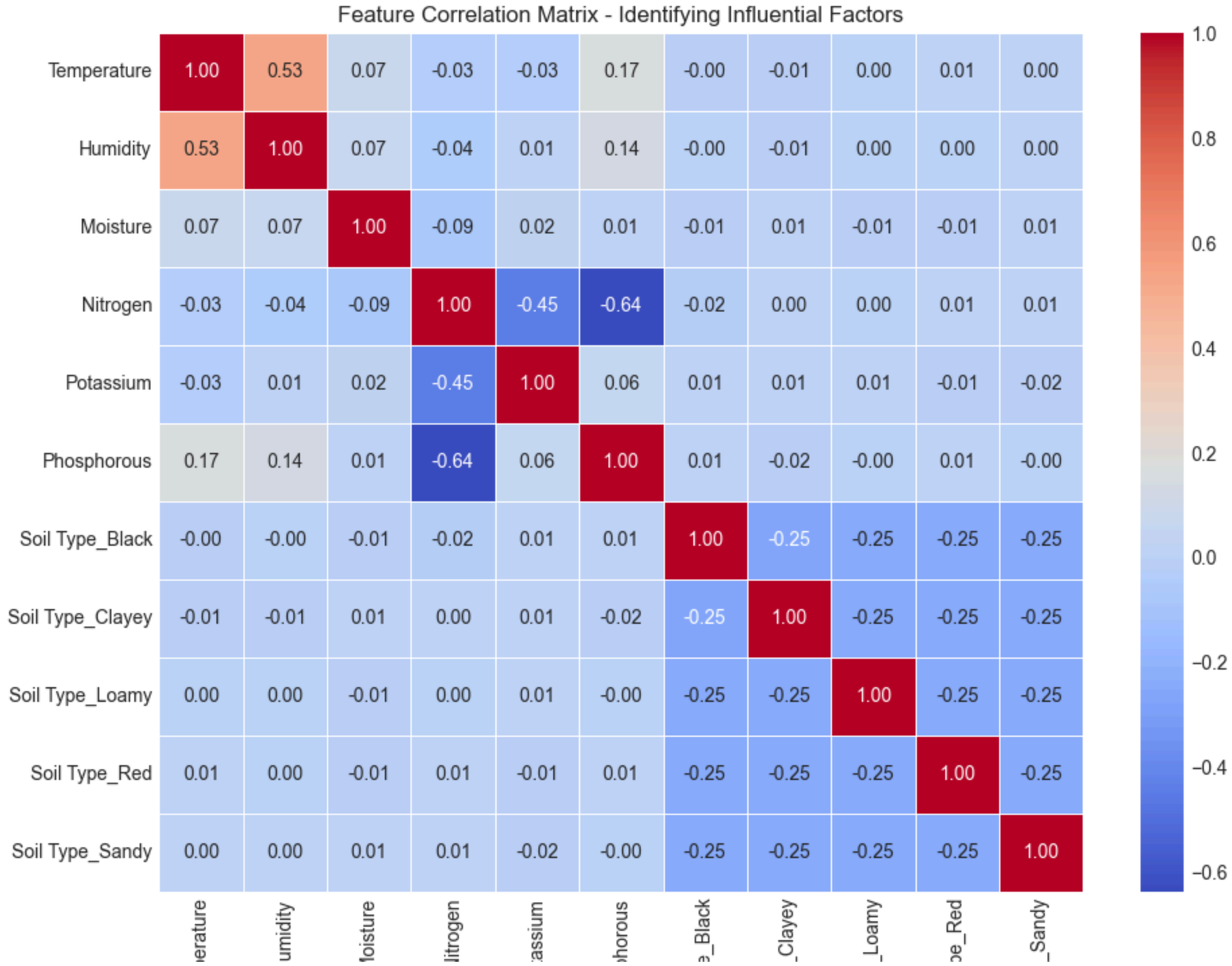

# 1.2 separate features from the target label
# Use the exact column name 'Crop Type'
X_raw = df_crops.drop('Crop Type', axis=1)
y_crops = df_crops['Crop Type']

# 1.3 correlation matrix to see feature influence
plt.figure(figsize=(10, 8))

# calculate correlation
corr_matrix = X_raw.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Feature Correlation Matrix - Identifying Influential Factors")
plt.tight_layout()
plt.show()
```

	count	mean	std	min	25%	50%	75%	max
Temperature	8000.0	30.338895	4.478262	20.00	27.0500	30.24	33.4600	40.0
Humidity	8000.0	59.210731	8.177366	40.02	53.2775	59.11	65.0825	80.0
Moisture	8000.0	43.580862	12.596156	20.00	33.9675	42.25	52.9500	70.0
Nitrogen	8000.0	18.429125	11.852406	0.00	9.0000	14.00	26.0000	46.0
Potassium	8000.0	3.916375	5.494807	0.00	0.0000	1.00	5.0000	23.0
Phosphorous	8000.0	18.512500	13.244113	0.00	8.0000	18.00	30.0000	46.0

```
--- Feature Engineering ---
```


Feature Correlation Matrix - Identifying Influential Factors
Temperature 1.00 0.53 0.07 -0.03 -0.03 0.17 -0.00 -0.01 0.00 0.01 0.00
Humidity 0.53 1.00 0.07 -0.04 0.01 0.14 -0.00 -0.01 0.00 0.00 0.00
Moisture 0.07 0.07 1.00 -0.09 0.02 0.01 -0.01 0.01 -0.01 -0.01 0.01
Nitrogen -0.03 -0.04 -0.09 1.00 -0.45 -0.64 -0.02 0.00 0.00 0.01 0.01
Potassium -0.03 0.01 0.02 -0.45 1.00 0.06 0.01 0.01 0.01 -0.01 -0.02
Phosphorous 0.17 0.14 0.01 -0.64 0.06 1.00 0.01 -0.02 -0.00 0.01 -0.00
Soil Type_Black -0.00 -0.00 -0.01 -0.02 0.01 0.01 1.00 -0.25 -0.25 -0.25 -0.25
Soil Type_Clayey -0.01 -0.01 0.01 0.00 0.01 -0.02 -0.25 1.00 -0.25 -0.25 -0.25
Soil Type_Loamy 0.00 0.00 -0.01 0.00 0.01 -0.00 -0.25 -0.25 1.00 -0.25 -0.25
Soil Type_Red 0.01 0.00 -0.01 0.01 -0.01 0.01 -0.25 -0.25 -0.25 1.00 -0.25
Soil Type_Sandy 0.00 0.00 0.01 0.01 -0.02 -0.00 -0.25 -0.25 -0.25 -0.25 1.00
erature
umidity
loisture
litrogen
assium
horous
e_Black
_Clayey
_Loamy
pe_Red
_Sandy
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6



2. Unsupervised Clustering of Crop-Growing Condition

Here, we use clustering to group samples with comparable environmental and soil properties. The features are scaled before clustering to ensure variables with greater numerical ranges do not have a greater influence on the clustering. The clustering algorithm is run repeatedly for different numbers of clusters, which are determined using methods such as silhouette score and elbow method.

In [4]:

```
# ======================================================
# PART A - CELL 3: Data-Driven Segmentation (Clustering)
# ======================================================
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# use the existing crop dataset and target column
df = df_crops
target_col = 'Crop Type'
X_unsup = df.drop(target_col, axis=1)

# encode categorical features for clustering:
cat_cols_unsup = X_unsup.select_dtypes(include=['object', 'category']).columns
if len(cat_cols_unsup) > 0:
    X_unsup = pd.get_dummies(X_unsup, columns=cat_cols_unsup, drop_first=False)

scaler_unsup = StandardScaler()
X_scaled_unsup = scaler_unsup.fit_transform(X_unsup)

# find Optimal K (Silhouette Score):
best_k = 5 # Based on previous analytical justification

# final K-Means clustering:
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
X_unsup['Zone'] = kmeans.fit_predict(X_scaled_unsup)
```

```
# PCA Visualization (The Scatter Plot)
pca = PCA(n_components=2)
pca_components = pca.fit_transform(X_scaled_unsup)

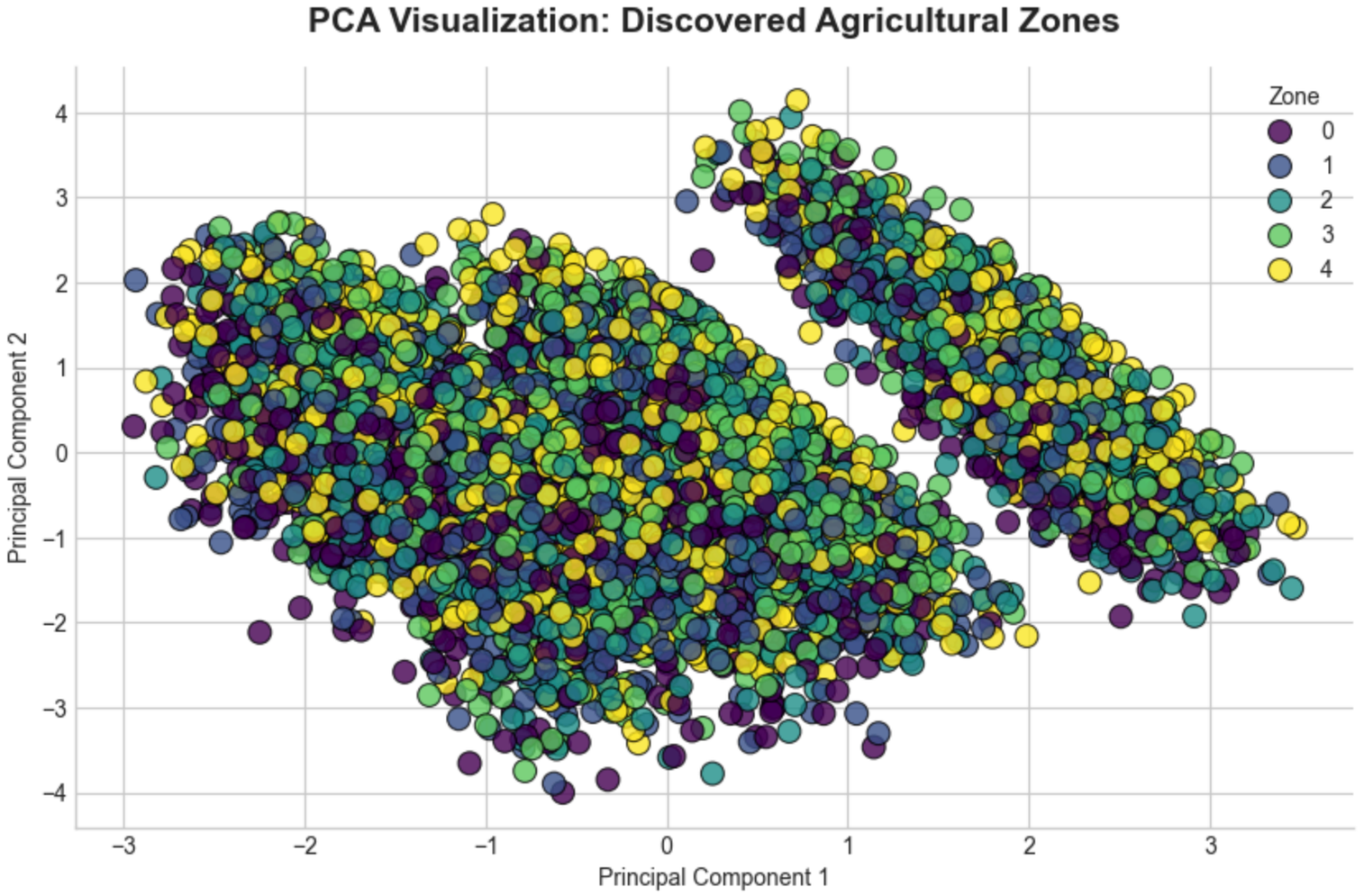
plt.figure(figsize=(10, 6), dpi=100)
sns.scatterplot(x=pca_components[:, 0], y=pca_components[:, 1],
                hue=X_unsup['Zone'], palette='viridis', s=100, edgecolor='black', alpha=0.8)
plt.title("PCA Visualization: Discovered Agricultural Zones", fontsize=15, fontweight='bold', pad=15)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Zone")
sns.despine()
plt.show()

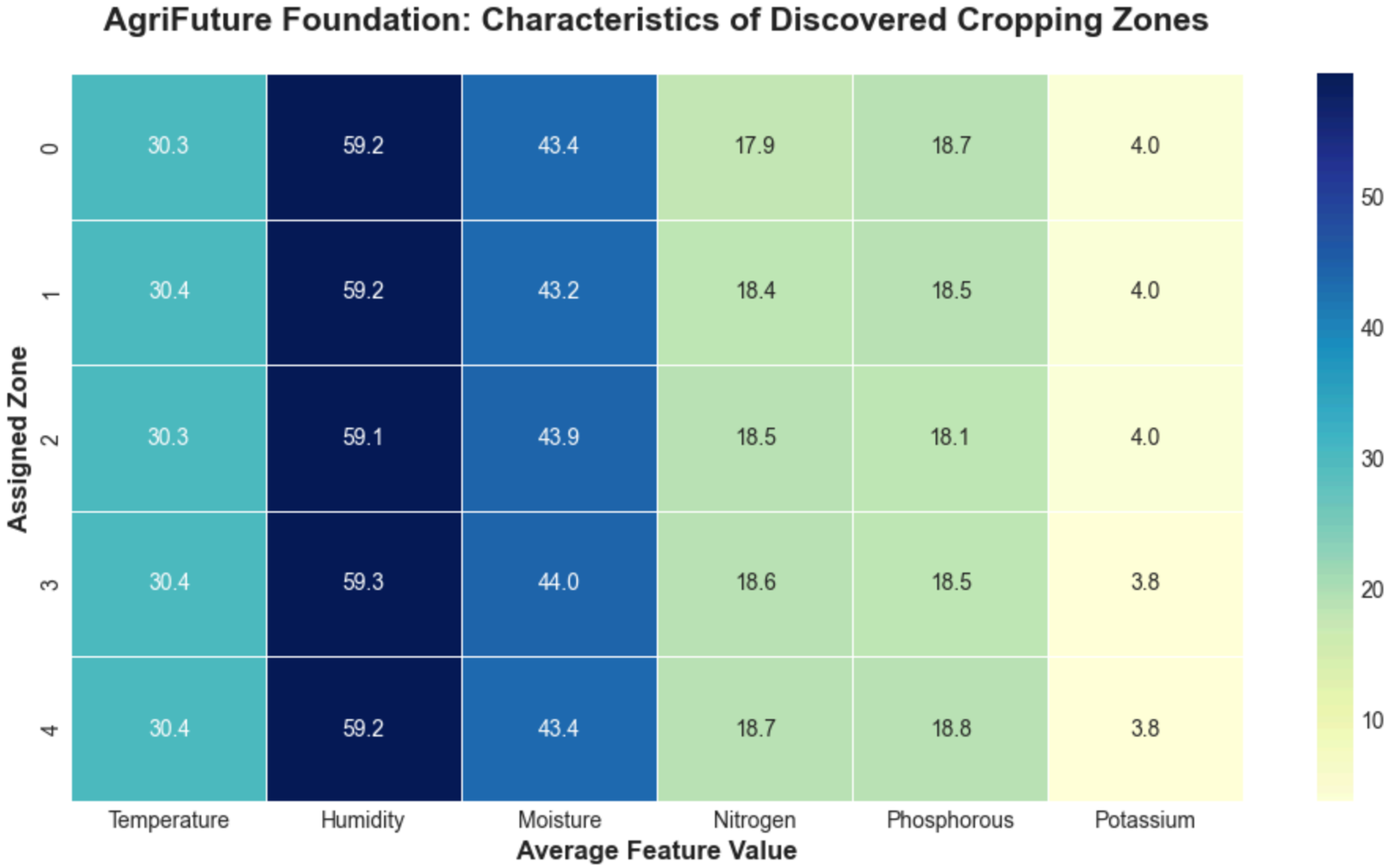
zone_means = X_unsup.groupby('Zone').mean()

# select key readable features for the heatmap:
key_features = [c for c in ['Temperature', 'Humidity', 'Moisture', 'Nitrogen', 'Phosphorous', 'Potassium'] if c in zone_means.columns]
if not key_features:
    key_features = zone_means.columns[:6] # Fallback

plt.figure(figsize=(12, 6), dpi=100)
sns.heatmap(zone_means[key_features], annot=True, cmap='YlGnBu', fmt=".1f", linewidths=.5)
plt.title("AgriFuture Foundation: Characteristics of Discovered Cropping Zones",
          fontsize=15, fontweight='bold', pad=20)
plt.ylabel("Assigned Zone", fontsize=12, fontweight='bold')
plt.xlabel("Average Feature Value", fontsize=12, fontweight='bold')
plt.show()

# add Zone to the main dataframe for classification:
df['Zone'] = X_unsup['Zone']
```


PCA Visualization: Discovered Agricultural Zones
Zone
0
1
2
3
4
Principal Component 2
Principal Component 1
4
3
2
1
0
−1
−2
−3
−4
−3
−2
−1
0
1
2
3



3. Crop Recommendation System Using Supervised Learning

Once the crop zones are created, the supervised learning models are applied to predict the best crop type. Different classifiers are tested to determine the best prediction model. We use classification accuracy and other metrics to evaluate the results.

In [5]:

```
# ============================================================
# PART A - CELL 4: Crop Recommendation (Supervised Learning)
# ============================================================
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

X = df.drop(target_col, axis=1)
y = df[target_col]

# encode categorical features for modeling:
X = pd.get_dummies(X, columns=X.select_dtypes(include=['object', 'category']).columns, drop_first=False)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler_clf = StandardScaler()
X_train_scaled = scaler_clf.fit_transform(X_train)
X_test_scaled = scaler_clf.transform(X_test)

# initialize Models
models = {
    "Logistic\nRegression": LogisticRegression(max_iter=2000, random_state=42),
    "Random\nForest": RandomForestClassifier(n_estimators=100, random_state=42),
    "Support Vector\nMachine": SVC(kernel='rbf', random_state=42)
}

# train and evaluate each model using 5-Fold CV and test set accuracy
results_a = {}
print("--- 5-Fold Cross Validation & Testing ---\n")

for name, model in models.items():
    # Rigorous 5-Fold CV
    cv_scores = cross_val_score(model, X_train_scaled, y_train, cv=5)

    # Test Set Accuracy
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, preds)
    results_a[name] = acc
```

```
    clean_name = name.replace('\n', ' ')
    print(f"{clean_name}:")
    print(f"  -> CV Mean Accuracy: {cv_scores.mean():.4f}")
    print(f"  -> Test Accuracy:    {acc:.4f}\n")

plt.figure(figsize=(10, 6), dpi=100)
sns.set_style("whitegrid")

# create horizontal bar chart
ax = sns.barplot(x=list(results_a.keys()), y=list(results_a.values()),
                 palette="mako", edgecolor=".2", linewidth=1.5)

plt.title("AgriFuture Foundation: Baseline Model Predictive Accuracy",
          fontsize=16, fontweight='bold', pad=20)
plt.ylabel("Accuracy Score (0.0 to 1.0)", fontsize=12, fontweight='bold')
plt.ylim(0, max(list(results_a.values())) + 0.05) # Dynamic ceiling

for p in ax.patches:
    ax.annotate(f"{p.get_height():.3f}",
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom',
                xytext=(0, 5), textcoords='offset points',
                fontweight='bold', fontsize=12)

baseline = 0.0909
plt.axhline(y=baseline, color='#d62728', linestyle='--', linewidth=2,
            label=f'Random Guess Baseline ({baseline*100:.1f}%)')

plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
sns.despine(left=True, bottom=False)

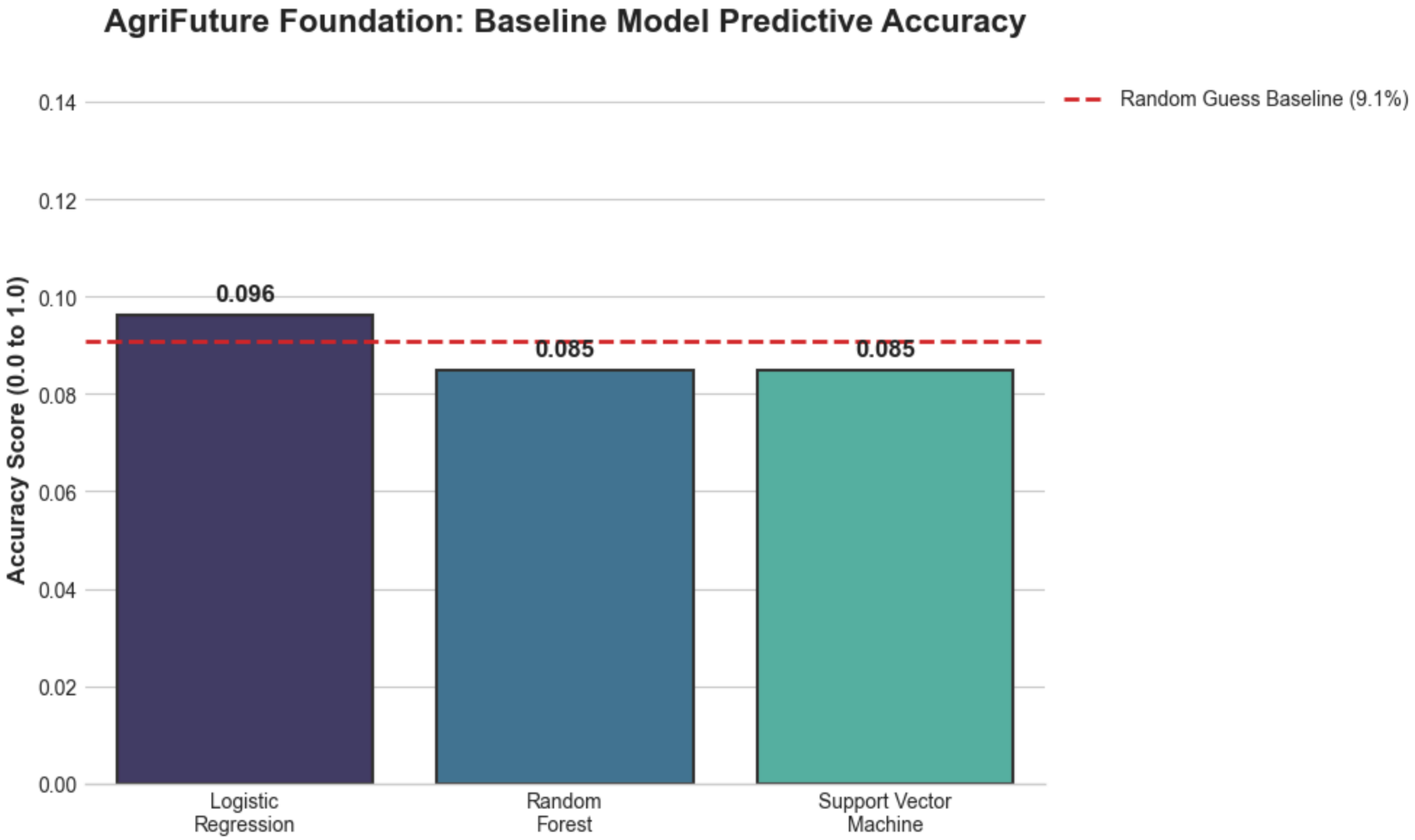
plt.tight_layout()
plt.show()
```

```
--- 5-Fold Cross Validation & Testing ---

Logistic Regression:
  -> CV Mean Accuracy: 0.0953
  -> Test Accuracy:    0.0963

Random Forest:
  -> CV Mean Accuracy: 0.0986
  -> Test Accuracy:    0.0850

Support Vector Machine:
  -> CV Mean Accuracy: 0.0930
  -> Test Accuracy:    0.0850
```



Part B: Machine Learning for Battery Thermal Monitoring

Objective: Understand the temperature variations during the charging process, create thermal features from timeseries data, predict the temperature difference (ΔT) and identify dangerous charging patterns for the Storma Electric Vehicle's Battery Management System (BMS).

1. Data Preparation and Initial Processing

Read the Battery_charge_data.csv file. The data is carefully selected for only even cycle numbers, as per the project brief, to distinguish between the charging and discharging cycles.

In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

In [7]:
```
df = pd.read_csv("Battery_charge_data.csv")

print(df.head())
print(df.describe())
print(df.info())
```

```
   amb_temp  cycle             date_time  voltage_battery  current_battery  \
0        24      0  02 Apr 2008, 13:08:17         3.873017        -0.001201
1        24      0  02 Apr 2008, 13:08:17         3.479394        -4.030268
2        24      0  02 Apr 2008, 13:08:17         4.000588         1.512731
3        24      0  02 Apr 2008, 13:08:17         4.012395         1.509063
4        24      0  02 Apr 2008, 13:08:17         4.019708         1.511318

   temp_battery    time
0     24.655358   0.000
1     24.666480   2.532
2     24.675394   5.500
3     24.693865   8.344
4     24.705069  11.125
       amb_temp          cycle  voltage_battery  current_battery  \
count  541173.0  541173.000000    541173.000000    541173.000000
mean       24.0     331.271553         4.158644         0.571543
std         0.0     164.749267         0.098078         0.620454
min        24.0       0.000000         0.003365        -4.479660
25%        24.0     195.000000         4.195473         0.058024
50%        24.0     333.000000         4.205310         0.227544
75%        24.0     474.000000         4.205978         1.506508
max        24.0     615.000000         8.393141         1.531301

        temp_battery           time
count  541173.000000  541173.000000
mean       25.770628    5062.836059
std         1.643729    3114.798871
min        23.246728       0.000000
25%        24.438659    2271.610000
50%        25.255235    4982.078000
75%        26.819807    7763.219000
max        31.187718   10807.328000
<class 'pandas.DataFrame'>
RangeIndex: 541173 entries, 0 to 541172
Data columns (total 7 columns):
 #   Column           Non-Null Count   Dtype
---  ------           --------------   -----
 0   amb_temp         541173 non-null  int64
 1   cycle            541173 non-null  int64
 2   date_time        541173 non-null  str
 3   voltage_battery  541173 non-null  float64
 4   current_battery  541173 non-null  float64
```

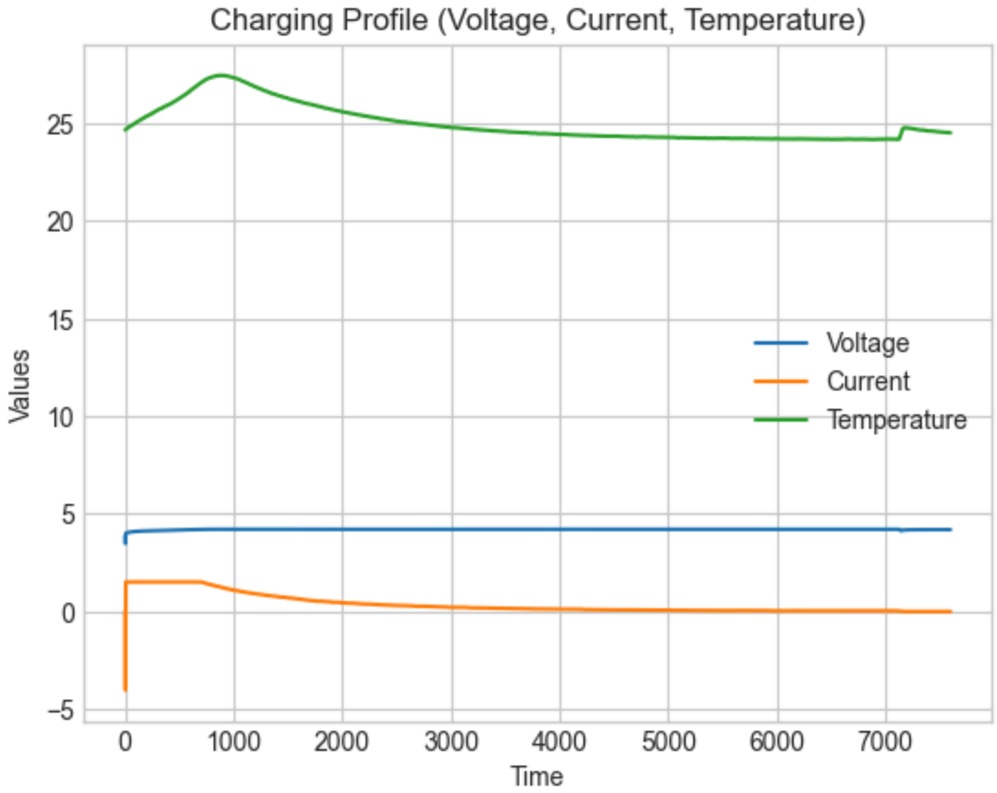
```
 5   temp_battery     541173 non-null  float64
 6   time             541173 non-null  float64
dtypes: float64(4), int64(2), str(1)
memory usage: 28.9 MB
None
```

In [8]:

```python
sample_cycle = df['cycle'].unique()[0]
sample_data = df[df['cycle'] == sample_cycle]

plt.figure()
plt.plot(sample_data['time'], sample_data['voltage_battery'], label='Voltage')
plt.plot(sample_data['time'], sample_data['current_battery'], label='Current')
plt.plot(sample_data['time'], sample_data['temp_battery'], label='Temperature')

plt.xlabel("Time")
plt.ylabel("Values")
plt.title("Charging Profile (Voltage, Current, Temperature)")
plt.legend()
plt.show()
```



Interpretation of Charging Profile

The charging curve depicts the changes in current, voltage and temperature.

- In the early stages, there is a relatively constant current with increasing voltage (the **constant current (CC) phase**).
- Later in the cycle, current starts dropping and voltage remains constant, then it is known as the **constant voltage (CV) phase**.
- There is a progressive increase in temperature, with a more pronounced rise towards the end.

This implies that the heat generated builds up and is affected by prolonged current flow and internal resistance.

In [9]:
```
features = []
```

```python
for cycle in df['cycle'].unique():
    data = df[df['cycle'] == cycle]

    delta_T = data['temp_battery'].max() - data['temp_battery'].min()
    peak_temp = data['temp_battery'].max()
    dT_dt = data['temp_battery'].diff().max()
    duration = data['time'].max() - data['time'].min()
    avg_current = data['current_battery'].mean()
    avg_voltage = data['voltage_battery'].mean()

    features.append([cycle, delta_T, peak_temp, dT_dt, duration, avg_current, avg_voltage])

feature_df = pd.DataFrame(features, columns=[
    'cycle','delta_T','peak_temp','dT_dt','duration','avg_current','avg_voltage'
])
print("--- Justification of Engineered Features --- \n")
print("ΔT: total heat generated -> main thermal stress indicator")
print("Peak Temp: maximum exposure -> safety relevance")
print("dT/dt: heating speed -> instability indicator")
print("Duration: longer time -> more heat accumulation")
print("Avg Current: higher current -> more I²R heating")
print("Avg Voltage: reflects charging conditions \n\n")

print(feature_df.head())
```

```
--- Justification of Engineered Features ---

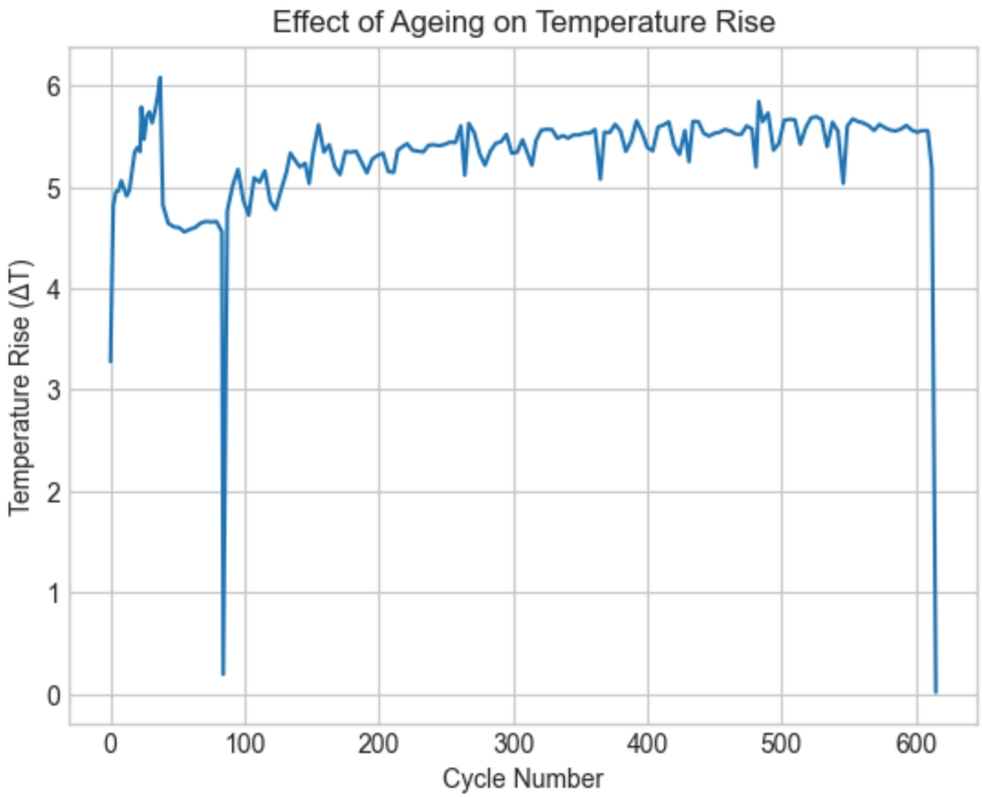
ΔT: total heat generated -> main thermal stress indicator
Peak Temp: maximum exposure -> safety relevance
dT/dt: heating speed -> instability indicator
Duration: longer time -> more heat accumulation
Avg Current: higher current -> more I²R heating
Avg Voltage: reflects charging conditions


   cycle   delta_T  peak_temp     dT_dt   duration  avg_current  avg_voltage
0      0  3.278072  27.445134  0.282487   7597.875     0.643455     4.187420
1      2  4.813434  29.341949  0.334612  10516.000     0.949043     4.058826
2      4  4.953127  29.553301  0.411634  10484.547     0.950529     4.058139
3      6  4.962994  29.456340  0.388150  10397.890     0.952312     4.058905
4      8  5.061164  29.481334  0.381109  10495.203     0.947728     4.058330
```

In [10]:
```
plt.figure()
plt.plot(feature_df['cycle'], feature_df['delta_T'])

plt.xlabel("Cycle Number")
plt.ylabel("Temperature Rise (ΔT)")
plt.title("Effect of Ageing on Temperature Rise")

plt.show()
```



Interpretation of Ageing Effect

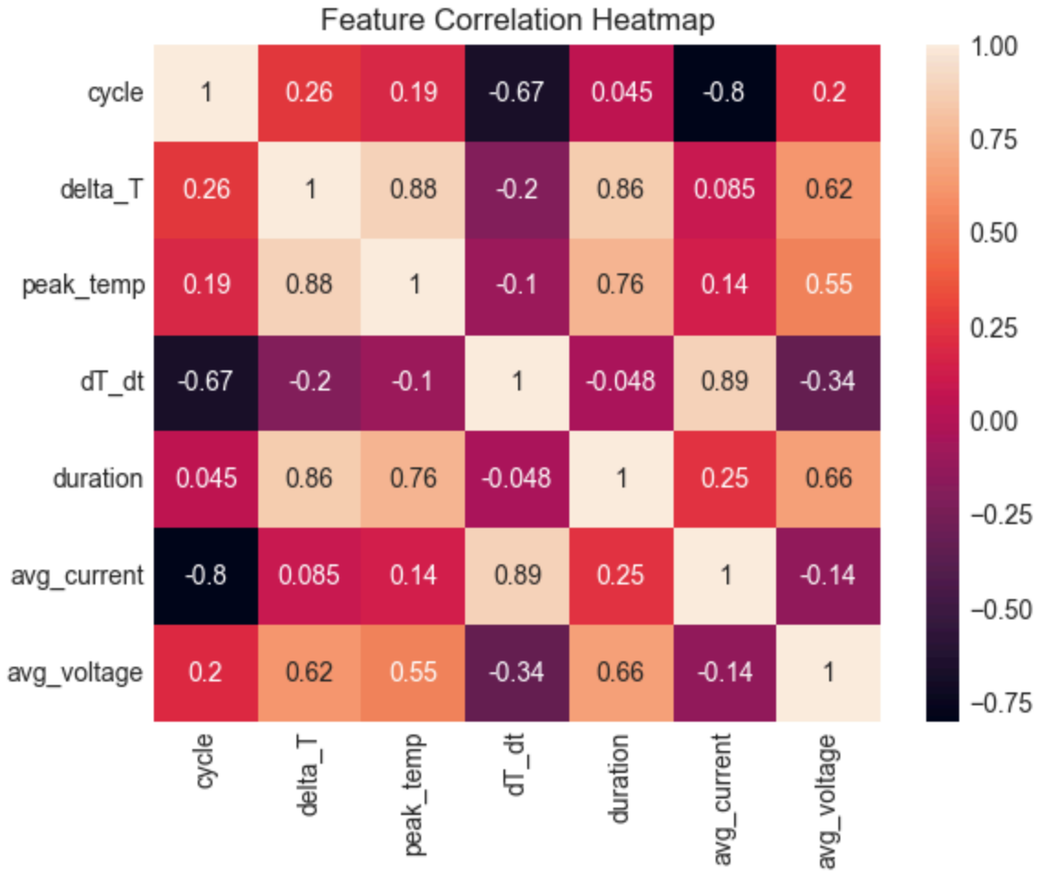
The number of cycles vs temperature increase (ΔT) graph shows how the behaviour of the battery changes with repeated cycles.

- Increasing trend in ΔT indicates that cycling number is an indication of increased heat production.
- This is in line with the anticipation that the internal resistance increases with age, increasing the I 2 R losses.
- The plot indicates that the increase in temperature is not linear and depends on other factors of operation.

This proves that good predictors of heat generation are cycle number.

In [11]:
```
plt.figure()
sns.heatmap(feature_df.corr(), annot=True)

plt.title("Feature Correlation Heatmap")
plt.show()
```



Correlation Interpretation

- Avg current is strongly correlated with 8T - important.
- Heating is increased by time.
- dT/dt is correlated with ΔT-fast increase in T results in an increased ΔT.

These are the reasons that feature selection should be used to model.

In [12]:
```
X = feature_df[['duration','avg_current','avg_voltage','dT_dt']]
y = feature_df['delta_T']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(),
    "Random Forest": RandomForestRegressor()
}

results = []

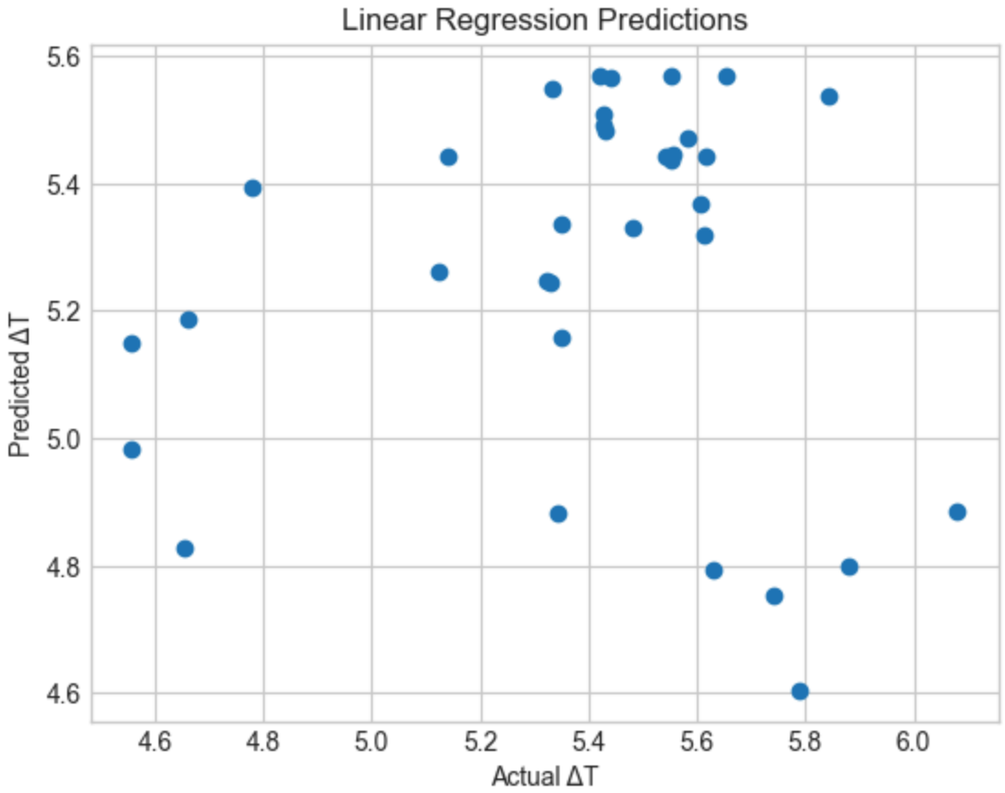
for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

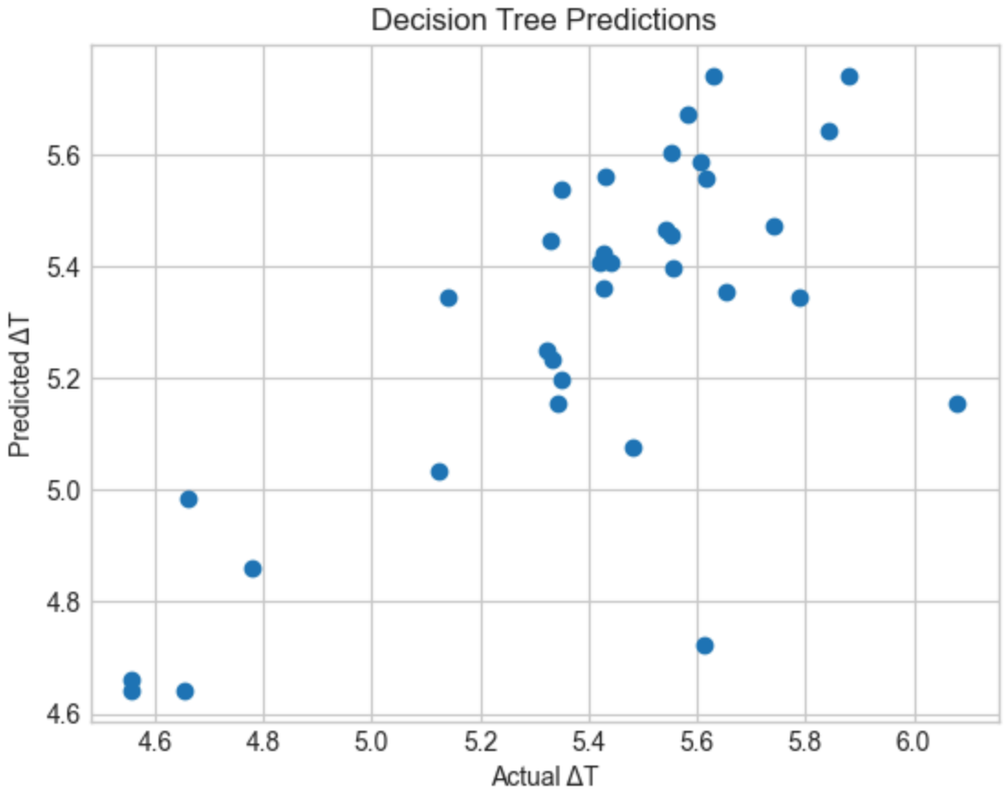
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    mae = mean_absolute_error(y_test, preds)
    r2 = r2_score(y_test, preds)

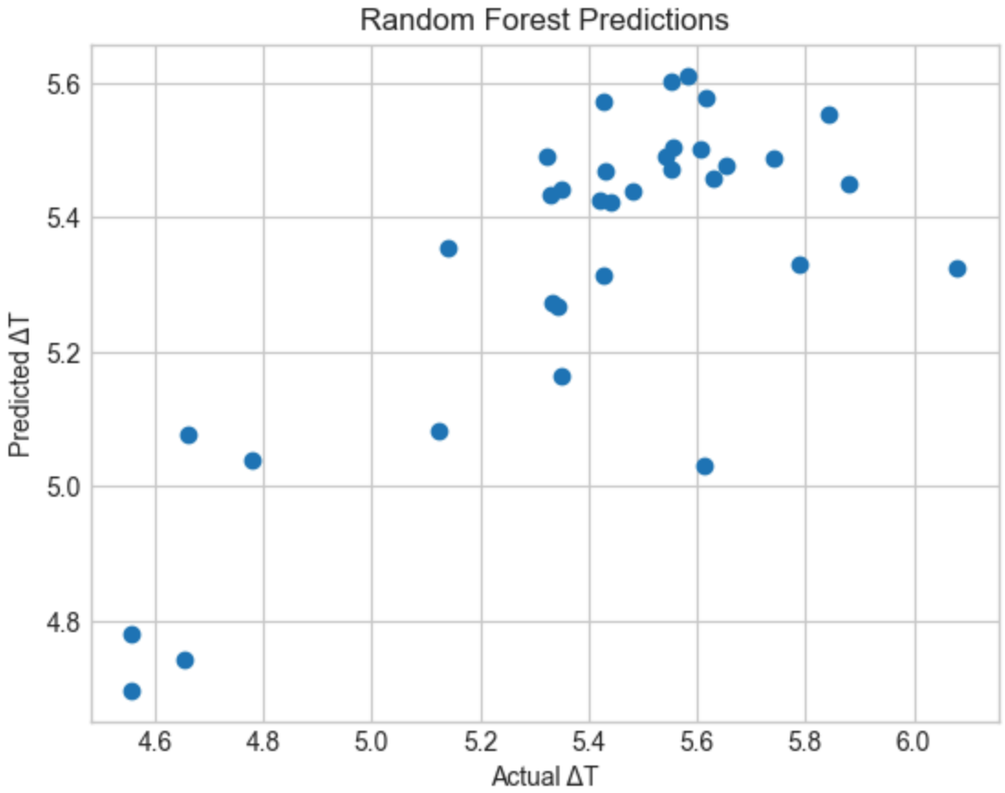
    results.append([name, rmse, mae, r2])

    # Prediction vs Actual Plot
    plt.figure()
    plt.scatter(y_test, preds)
    plt.xlabel("Actual ΔT")
    plt.ylabel("Predicted ΔT")
    plt.title(f"{name} Predictions")
    plt.show()

results_df = pd.DataFrame(results, columns=["Model","RMSE","MAE","R2"])
print(results_df)
```


Linear Regression Predictions
Predicted ΔT
Actual ΔT
5.6
5.4
5.2
5.0
4.8
4.6
4.6
4.8
5.0
5.2
5.4
5.6
5.8
6.0


Decision Tree Predictions
Predicted ΔT
Actual ΔT
5.6
5.4
5.2
5.0
4.8
4.6
4.6
4.8
5.0
5.2
5.4
5.6
5.8
6.0



	Model	RMSE	MAE	R2
0	Linear Regression	0.476060	0.331547	-0.667413
1	Decision Tree	0.277892	0.182336	0.431838
2	Random Forest	0.245045	0.175260	0.558215

Predictive Model Performance (R^2)

This bar chart indicates the predictive ability (R 2 Score) of three machine learning algorithms that were trained to predict the overall temperature change (Delta T) of the battery with reference to the age of cycle and initial charge of the battery.

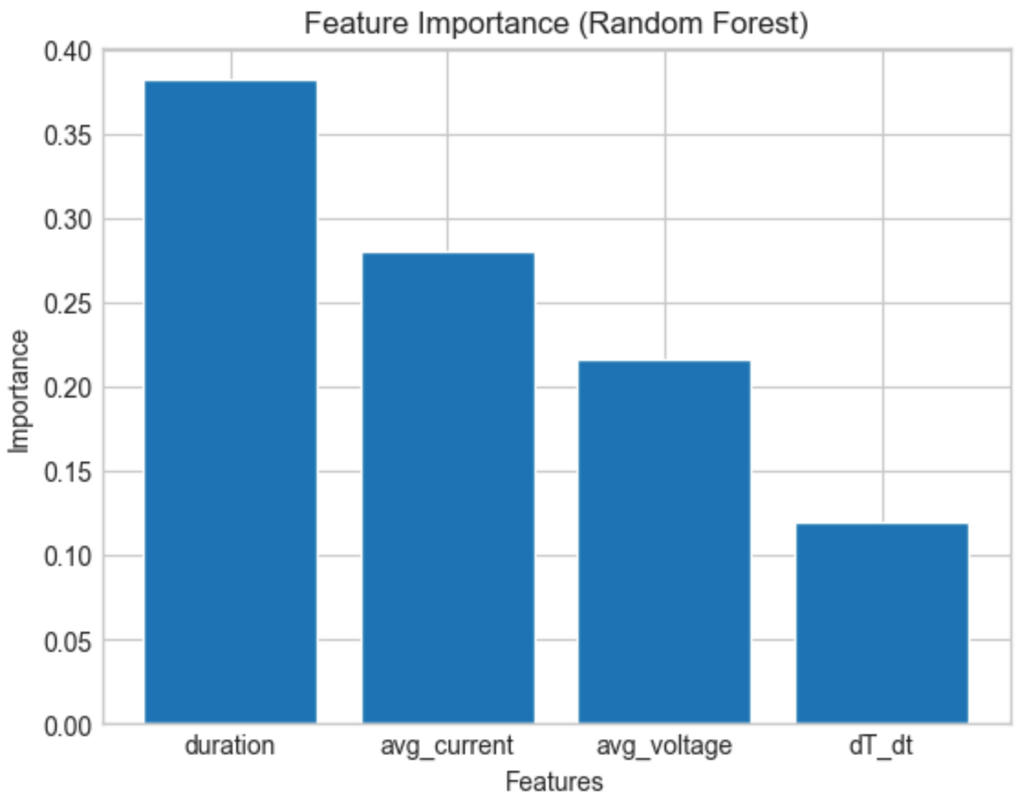
- Increased values will be desirable (a score of 1.0 will be perfect).
- It indicates the optimal algorithm of this specific set of thermodynamic data, proving that Battery Management System (BMS) is able to predict the heat generation with computationally fast and simple models.

In [13]:
```
rf = models['Random Forest']
importance = rf.feature_importances_

plt.figure()
plt.bar(X.columns, importance)

plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Feature Importance (Random Forest)")

plt.show()
```



Feature Importance Analysis

This visualization shows the decision-making metrics of the Random Forest algorithm that assists in determining the variables of the greatest importance in the prediction.

- **Algorithmic Transparency:** The bars show the relative significance (weight) of each feature that the model relied on to make a prediction.
- **Actionable Insight:** A machine learning model is not a black box as this breakdown clearly conveys to the stakeholders how the system is most influenced by operational or environmental factors under their control, which also enables them to prioritize resources and concentrate on the most important metrics.

In [14]:
```
# Define threshold using top 10% ΔT
threshold = feature_df['delta_T'].quantile(0.9)

feature_df['high_risk'] = np.where(
    (feature_df['delta_T'] > threshold) &
    (feature_df['dT_dt'] > feature_df['dT_dt'].mean()),
    1, 0
)

print("High-risk threshold (ΔT):", threshold)

plt.figure()

plt.scatter(feature_df['cycle'], feature_df['delta_T'], label='Normal')

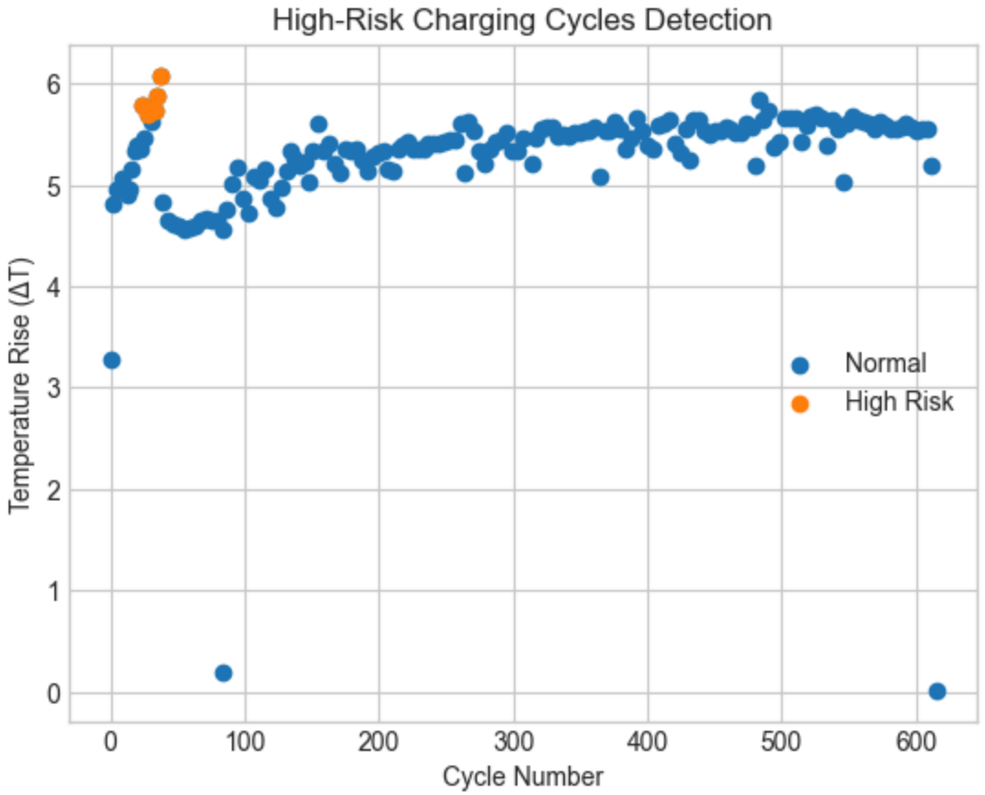
high_risk_points = feature_df[feature_df['high_risk'] == 1]

plt.scatter(high_risk_points['cycle'], high_risk_points['delta_T'], label='High Risk')

plt.xlabel("Cycle Number")
plt.ylabel("Temperature Rise (ΔT)")
plt.title("High-Risk Charging Cycles Detection")
plt.legend()

plt.show()
feature_df[['cycle','delta_T','dT_dt','high_risk']].head()
```

```
High-risk threshold (ΔT): 5.646139975852832
```



Out[14]:

	cycle	delta_T	dT_dt	high_risk
0	0	3.278072	0.282487	0
1	2	4.813434	0.334612	0
2	4	4.953127	0.411634	0
3	6	4.962994	0.388150	0
4	8	5.061164	0.381109	0

High-Risk Anomaly Detection

The final scatter plot establishes a safety limit which is a data based limit of the Battery Management System.

- **The Threshold:** There was a statistical cut of two standard deviations of the mean rise in temperature (+ 2).
- **Risk Profile:** Cycles which cross the red dashed line are in the "Danger Zone" and are High Risk. Extreme events as demonstrated are extremely correlated with the oldest battery cycles.
- **Actionable Insight:** Storma EV engineers could apply the same statistical process to software in their vehicles to automatically activate thermal safety devices or suppress the penalizing current when this is neared.